AIML Internal Assessment

Feature Engineering and Data Preprocessing on a Web-Scraped E-Commerce Dataset

Dataset Source: Web Scraper E-Commerce Test Site
Tools: Python, Requests, BeautifulSoup, Pandas, NumPy, Matplotlib, Seaborn, Scikit-Learn

Name: Dhruvesh Hemraj Biraris
PRN: 202401040239
Division: A
Roll No.: 55
Subject: Artificial Intelligence and Machine Learning

SECTION 1: Project Introduction

1.1 Problem Statement

E-commerce websites contain product information such as names, prices, categories, and descriptions. This information often needs cleaning before analysis. In this assignment, product data is collected from a public e-commerce test website, cleaned, analyzed, and prepared for machine learning.

1.2 Objectives

1. Collect product records using web scraping.
2. Store the data in a Pandas DataFrame and CSV file.
3. Explore the data and identify quality problems.
4. Clean text, duplicates, missing values, and data types.
5. Detect and treat price outliers.
6. Create useful features from existing columns.
7. Apply feature scaling and visualize patterns.
8. Train a basic machine-learning model when the data supports it.
9. Save and verify the final dataset.

1.3 Feature Engineering

Feature engineering creates useful information from existing columns. We create numeric prices, description lengths, category codes, price groups, and a premium-product flag.

1.4 Data Preprocessing

Preprocessing includes cleaning text, handling missing values, removing duplicates, converting data types, treating outliers, and preparing features for analysis.

SECTION 2: Web Scraping

2.1 Introduction

Web scraping collects information from web pages using code. Requests accesses the page, and BeautifulSoup extracts product details. The scraper checks multiple pages and keeps unique product URLs. The number of products depends on what the website makes available.

In [2]:
```
import re
import time
import os
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from bs4 import BeautifulSoup
from urllib.parse import urljoin
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classificat

pd.set_option("display.max_columns", None)

HEADERS = {"User-Agent": "Mozilla/5.0 (compatible; StudentDataAssignment/
TARGET_ROWS = 200
REQUEST_TIMEOUT = 20

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

2.2 Checking the Website

First, check whether the test website is reachable.

In [3]:
```
test_url = "https://webscraper.io/test-sites/e-commerce/static"
try:
    response = requests.get(test_url, headers=HEADERS, timeout=REQUEST_TI
    print("Status code:", response.status_code)
    print("HTML length:", len(response.text))
    page_title = BeautifulSoup(response.text, "html.parser").title
    print("Page title:", page_title.get_text(strip=True) if page_title el
except requests.RequestException as exc:
    print("Website check failed:", exc)
```

```
Status code: 200
HTML length: 30424
Page title: Static | Web Scraper Test Sites
```

2.3 Collecting Product Records

This cell collects unique product cards from the public test site's category pages. It stops at 200 records if enough are available. If fewer are accessible, it reports the actual count rather than inventing data.

In [4]:
```
def clean_text(value):
    if value is None:
        return ""
    return re.sub(r"\\s+", " ", str(value)).strip()

def parse_price(value):
    if value is None:
        return np.nan
    match = re.search(r"\\d[\\d,]*(?:\\.\\d+)?", str(value))
    return float(match.group(0).replace(",", "")) if match else np.nan

def add_product(records, seen_urls, name, price, category, description, p
    name = clean_text(name)
    product_url = clean_text(product_url)
    if not name or not product_url or product_url in seen_urls:
        return
    seen_urls.add(product_url)
    records.append({
        "Product_Name": name,
        "Price": clean_text(price),
        "Category": clean_text(category) or "Unknown",
        "Description": clean_text(description),
        "Availability": "Available",
        "Product_URL": product_url,
```

```
        "Source": source
    })

def scrape_webscraper_site(target=200):
    base = "https://webscraper.io/test-sites/e-commerce/static"
    category_pages = [
        ("Computers - Laptops", f"{base}/computers/laptops"),
        ("Computers - Tablets", f"{base}/computers/tablets"),
        ("Phones", f"{base}/phones"),
        ("Computers", f"{base}/computers")
    ]
    records, seen_urls = [], set()

    for category_name, category_url in category_pages:
        for page_num in range(1, 31):
            if len(records) >= target:
                break
            page_url = category_url if page_num == 1 else f"{category_url
            try:
                res = requests.get(page_url, headers=HEADERS, timeout=REQ
                if res.status_code != 200:
                    print(f"Stopping this category at status {res.status_
                    break
                soup = BeautifulSoup(res.text, "html.parser")
                cards = soup.select("div.thumbnail")
                if not cards:
                    break

                old_count = len(records)
                for card in cards:
                    title_tag = card.select_one("a.title")
                    if title_tag is None:
                        continue
                    price_tag = card.select_one("h4.price")
                    desc_tag = card.select_one("p.description")
                    product_url = urljoin(page_url, title_tag.get("href",
                    add_product(
                        records, seen_urls,
                        title_tag.get_text(" ", strip=True),
                        price_tag.get_text(" ", strip=True) if price_tag
                        category_name,
                        desc_tag.get_text(" ", strip=True) if desc_tag el
                        product_url,
                        "Web Scraper Test Site"
                    )
                    if len(records) >= target:
                        break

                print(f"{category_name} page {page_num}: {len(records)} u
                if len(records) == old_count:
                    break
                time.sleep(0.2)
            except requests.RequestException as exc:
                print(f"Could not fetch {page_url}: {exc}")
```

```
                break
        if len(records) >= target:
            break

    return pd.DataFrame(records)

raw_df = scrape_webscraper_site(TARGET_ROWS)
print("\nRaw dataset shape:", raw_df.shape)
display(raw_df.head())
```

```
Computers - Laptops page 1: 6 unique products
Computers - Laptops page 2: 12 unique products
Computers - Laptops page 3: 18 unique products
Computers - Laptops page 4: 24 unique products
Computers - Laptops page 5: 30 unique products
Computers - Laptops page 6: 36 unique products
Computers - Laptops page 7: 42 unique products
Computers - Laptops page 8: 48 unique products
Computers - Laptops page 9: 54 unique products
Computers - Laptops page 10: 60 unique products
Computers - Laptops page 11: 66 unique products
Computers - Laptops page 12: 72 unique products
Computers - Laptops page 13: 78 unique products
Computers - Laptops page 14: 84 unique products
Computers - Laptops page 15: 90 unique products
Computers - Laptops page 16: 96 unique products
Computers - Laptops page 17: 102 unique products
Computers - Laptops page 18: 108 unique products
Computers - Laptops page 19: 114 unique products
Computers - Laptops page 20: 117 unique products
Computers - Tablets page 1: 123 unique products
Computers - Tablets page 2: 129 unique products
Computers - Tablets page 3: 135 unique products
Computers - Tablets page 4: 138 unique products
Phones page 1: 141 unique products
Phones page 2: 144 unique products
Phones page 3: 146 unique products
Phones page 4: 146 unique products
Computers page 1: 146 unique products

Raw dataset shape: (146, 7)
```

	Product_Name	Price	Category	Description	Availability	Product_URL	Source
0	Packard 255 G2	$416.99	Computers - Laptops	15.6", AMD E2-3800 1.3GHz, 4GB, 500GB, Windows...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
1	Aspire E1-510	$306.99	Computers - Laptops	15.6", Pentium N3520 2.16GHz, 4GB, 500GB, Linux	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
2	ThinkPad T540p	$1178.99	Computers - Laptops	15.6", Core i5-4200M, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
3	ProBook	$739.99	Computers - Laptops	14", Core i5 2.6GHz, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
4	ThinkPad X240	$1311.99	Computers - Laptops	12.5", Core i5-4300U, 8GB, 240GB SSD, Win7 Pro...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site

2.4 Checking the Record Count

The next cell checks whether the scraper reached the target. A small count means the website did not expose enough unique products through these pages.

In [5]:
```
if len(raw_df) >= TARGET_ROWS:
    print(f"Target reached: {len(raw_df)} unique products.")
else:
    print(f"Collected {len(raw_df)} unique products; target was {TARGET_R
    print("No rows have been duplicated or fabricated. An additional appr
```

```
Collected 146 unique products; target was 200.
No rows have been duplicated or fabricated. An additional approved source
may be needed for 200 real products.
```

SECTION 3: Raw Dataset Creation

3.1 Saving the Raw Data

The original scraped records are saved separately so they remain available after cleaning.

In [6]:
```
df_raw = raw_df.copy()
df_raw.to_csv("/content/raw_ecommerce_data.csv", index=False)
print("Saved raw dataset: /content/raw_ecommerce_data.csv")
print("Rows:", len(df_raw))
print("Columns:", len(df_raw.columns))
display(df_raw.head())
```

```
Saved raw dataset: /content/raw_ecommerce_data.csv
Rows: 146
Columns: 7
```

	Product_Name	Price	Category	Description	Availability	Product_URL	Source
0	Packard 255 G2	$416.99	Computers - Laptops	15.6", AMD E2-3800 1.3GHz, 4GB, 500GB, Windows...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
1	Aspire E1-510	$306.99	Computers - Laptops	15.6", Pentium N3520 2.16GHz, 4GB, 500GB, Linux	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
2	ThinkPad T540p	$1178.99	Computers - Laptops	15.6", Core i5-4200M, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
3	ProBook	$739.99	Computers - Laptops	14", Core i5 2.6GHz, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
4	ThinkPad X240	$1311.99	Computers - Laptops	12.5", Core i5-4300U, 8GB, 240GB SSD, Win7 Pro...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site

SECTION 4: Exploratory Data Analysis

4.1 Understanding the Dataset

We inspect the size, columns, data types, and sample records.

In [7]:
```
print("Dataset shape:", df_raw.shape)
print("Columns:", df_raw.columns.tolist())
print("\nData types:")
print(df_raw.dtypes)
df_raw.info()
display(df_raw.head())
```

```
Dataset shape: (146, 7)
Columns: ['Product_Name', 'Price', 'Category', 'Description', 'Availabili
ty', 'Product_URL', 'Source']

Data types:
Product_Name    object
Price           object
Category        object
Description     object
Availability    object
Product_URL     object
Source          object
dtype: object
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 146 entries, 0 to 145
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Product_Name  146 non-null    object
 1   Price         146 non-null    object
 2   Category      146 non-null    object
 3   Description   146 non-null    object
 4   Availability  146 non-null    object
 5   Product_URL   146 non-null    object
 6   Source        146 non-null    object
dtypes: object(7)
memory usage: 8.1+ KB
```

	Product_Name	Price	Category	Description	Availability	Product_URL	Source
0	Packard 255 G2	$416.99	Computers - Laptops	15.6", AMD E2-3800 1.3GHz, 4GB, 500GB, Windows...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
1	Aspire E1-510	$306.99	Computers - Laptops	15.6", Pentium N3520 2.16GHz, 4GB, 500GB, Linux	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
2	ThinkPad T540p	$1178.99	Computers - Laptops	15.6", Core i5-4200M, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
3	ProBook	$739.99	Computers - Laptops	14", Core i5 2.6GHz, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
4	ThinkPad X240	$1311.99	Computers - Laptops	12.5", Core i5-4300U, 8GB, 240GB SSD, Win7 Pro...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site

4.2 Statistical Summary

This summary gives an overview of the collected fields.

In [8]:
```
display(df_raw.describe(include="all").T)
```

	count	unique	top	freq
Product_Name	146	77	Dell Latitude...	13
Price	146	134	$899.99	3
Category	146	3	Computers - Laptops	117
Description	146	145	Acer Aspire 3 A315-31 Black, 15.6" HD, Pentium...	2
Availability	146	1	Available	146
Product_URL	146	146	https://webscraper.io/test-sites/e-commerce/st...	1
Source	146	1	Web Scraper Test Site	146

SECTION 5: Data Quality Problems

5.1 Checking Data Quality

We check missing values, duplicate records, blank names, and repeated URLs before cleaning.

In [9]:
```
print("Missing values:")
display(df_raw.isna().sum().to_frame("Missing_Count"))
print("Duplicate rows:", df_raw.duplicated().sum())
if "Product_Name" in df_raw:
    print("Blank product names:", df_raw["Product_Name"].fillna("").str.s
if "Product_URL" in df_raw:
    print("Duplicate URLs:", df_raw["Product_URL"].duplicated().sum())
```

```
Missing values:
```

	Missing_Count
Product_Name	0
Price	0
Category	0
Description	0
Availability	0
Product_URL	0
Source	0

```
Duplicate rows: 0
Blank product names: 0
Duplicate URLs: 0
```

SECTION 6: Data Cleaning

6.1 Cleaning Text and Duplicates

We create a fresh working DataFrame from the raw data. This prevents later cells from accidentally starting with an empty DataFrame.

In [10…
```
df = df_raw.copy()

required_columns = [
    "Product_Name", "Price", "Category", "Description",
    "Availability", "Product_URL", "Source"
```

```
]
for col in required_columns:
    if col not in df.columns:
        df[col] = ""

for col in required_columns:
    df[col] = df[col].fillna("").astype(str).map(clean_text)

df = df[df["Product_Name"].str.len() > 0].copy()
df = df.drop_duplicates(subset=["Product_URL"])
df["Category"] = df["Category"].replace("", "Unknown")
df["Description"] = df["Description"].replace("", "No description provide
df["Availability"] = df["Availability"].replace("", "Unknown")

print("Rows after text cleaning and deduplication:", len(df))
display(df.head())
```

```
Rows after text cleaning and deduplication: 146
```

	Product_Name	Price	Category	Description	Availability	Product_URL	Source
0	Packard 255 G2	$416.99	Computers - Laptops	15.6", AMD E2-3800 1.3GHz, 4GB, 500GB, Windows...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
1	Aspire E1-510	$306.99	Computers - Laptops	15.6", Pentium N3520 2.16GHz, 4GB, 500GB, Linux	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
2	ThinkPad T540p	$1178.99	Computers - Laptops	15.6", Core i5-4200M, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
3	ProBook	$739.99	Computers - Laptops	14", Core i5 2.6GHz, 4GB, 500GB, Win7 Pro 64bit	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site
4	ThinkPad X240	$1311.99	Computers - Laptops	12.5", Core i5-4300U, 8GB, 240GB SSD, Win7 Pro...	Available	https://webscraper.io/test-sites/e-commerce/st...	Web Scraper Test Site

6.2 Converting Price

The website stores prices as text. We extract the numeric amount into `Price_Numeric`. Rows are removed only if a price cannot be interpreted as a

number.

In [11…
```
df["Price_Numeric"] = df["Price"].apply(parse_price)
invalid_price_count = int(df["Price_Numeric"].isna().sum())
print("Invalid or missing prices:", invalid_price_count)

df = df.dropna(subset=["Price_Numeric"]).copy()
df["Price_Numeric"] = pd.to_numeric(df["Price_Numeric"], errors="coerce")
df = df.dropna(subset=["Price_Numeric"]).copy()

print("Rows with valid prices:", len(df))
display(df[["Product_Name", "Price", "Price_Numeric"]].head())
```

```
Invalid or missing prices: 146
Rows with valid prices: 0
```

Product_Name	Price	Price_Numeric

SECTION 7: Outlier Detection and Treatment

7.1 Detecting Outliers

The IQR method identifies unusually low or high prices. The cell safely handles an empty dataset so later cells can show a useful message instead of crashing.

In [12…
```
if df.empty:
    print("No valid product records remain. Check the raw dataset and pri
    Q1 = Q3 = IQR = lower_bound = upper_bound = np.nan
    df["Price_Treated"] = pd.Series(dtype=float)
else:
    Q1 = df["Price_Numeric"].quantile(0.25)
    Q3 = df["Price_Numeric"].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = max(0, Q1 - 1.5 * IQR)
    upper_bound = Q3 + 1.5 * IQR
    outlier_mask = (df["Price_Numeric"] < lower_bound) | (df["Price_Numer
    print("Q1:", Q1, "| Q3:", Q3, "| IQR:", IQR)
    print("Outliers detected:", int(outlier_mask.sum()))
    df["Price_Treated"] = df["Price_Numeric"].clip(lower=lower_bound, upp
```

```
No valid product records remain. Check the raw dataset and price conversi
on before continuing.
```

7.2 Treating Outliers

Extreme prices are capped at the IQR limits for analysis. The original text price and numeric price are retained.

In [13…
```
if not df.empty:
    display(df[["Product_Name", "Price_Numeric", "Price_Treated"]].head()
    print("Price treatment completed.")
else:
    print("Price treatment skipped because there are no valid records.")
```

```
Price treatment skipped because there are no valid records.
```

SECTION 8: Feature Engineering

8.1 Creating New Features

We create description length, product-name length, availability flag, price category, and premium-product status.

In [14…
```
if not df.empty:
    df["Description_Length"] = df["Description"].str.len()
    df["Product_Name_Length"] = df["Product_Name"].str.len()
    df["Availability_Flag"] = df["Availability"].str.lower().eq("availabl

    # Quantile-based groups adapt to the observed price range.
    if df["Price_Treated"].nunique() >= 3:
        df["Price_Category"] = pd.qcut(
            df["Price_Treated"].rank(method="first"),
            q=3, labels=["Low", "Medium", "High"]
        ).astype(str)
    else:
        df["Price_Category"] = "Medium"

    df["Premium_Product"] = (df["Price_Treated"] > df["Price_Treated"].me
    df["Category_Code"] = df["Category"].astype("category").cat.codes
else:
    for col in ["Description_Length", "Product_Name_Length", "Availabilit
                "Price_Category", "Premium_Product", "Category_Code"]:
        df[col] = pd.Series(dtype=float)

display(df.head())
```

Product_Name	Price	Category	Description	Availability	Product_URL	Source	Price_Numer

8.2 Encoding Categories

Each product category is mapped to a numeric code while the original text category is retained.

In [15…
```
if not df.empty:
    df["Category_Code"] = df["Category"].astype("category").cat.codes
    display(df[["Category", "Category_Code"]].drop_duplicates().head(15))
else:
    print("Category encoding skipped because there are no product rows.")
```

```
Category encoding skipped because there are no product rows.
```

SECTION 9: Feature Scaling

9.1 Standardization

Standardization puts numeric features on a comparable scale. The cell checks for empty data before calling the scaler.

In [16…
```
scale_features = ["Price_Treated", "Description_Length", "Product_Name_Le

if df.empty:
    print("Standardization skipped: df has zero rows. Return to Sections
else:
    scaler_input = df[scale_features].replace([np.inf, -np.inf], np.nan)
    scaler_input = scaler_input.fillna(scaler_input.median()).fillna(0)
    standard_scaler = StandardScaler()
    standard_values = standard_scaler.fit_transform(scaler_input)

    for i, col in enumerate(scale_features):
        df[col + "_Standardized"] = standard_values[:, i]
    print("Standardization completed.")
    display(df[[col + "_Standardized" for col in scale_features]].head())
```

```
Standardization skipped: df has zero rows. Return to Sections 2 and 6 to
check collection and cleaning.
```

9.2 Min-Max Normalization

Min-Max scaling transforms numeric values to a range between 0 and 1.

In [17…
```
if df.empty:
    print("Normalization skipped: there are no rows to scale.")
else:
    minmax_scaler = MinMaxScaler()
```

```
    norm_input = df[["Price_Treated", "Description_Length"]].replace([np.
    norm_input = norm_input.fillna(norm_input.median()).fillna(0)
    normalized = minmax_scaler.fit_transform(norm_input)
    df["Price_Normalized"] = normalized[:, 0]
    df["Description_Normalized"] = normalized[:, 1]
    print("Min-Max normalization completed.")
    display(df[["Price_Normalized", "Description_Normalized"]].head())
```

```
Normalization skipped: there are no rows to scale.
```

SECTION 10: Visualization

Graphs help us understand price patterns and the distribution of products across categories.

In [18…

```
if df.empty:
    print("Price histogram skipped: no product data is available.")
else:
    plt.figure(figsize=(8, 5))
    sns.histplot(df["Price_Treated"], kde=len(df) > 1)
    plt.title("Product Price Distribution")
    plt.xlabel("Treated Price")
    plt.ylabel("Number of Products")
    plt.tight_layout()
    plt.show()
```

```
Price histogram skipped: no product data is available.
```

In [19…

```
if df.empty:
    print("Category chart skipped: no product data is available.")
else:
    category_counts = df["Category"].value_counts().head(15)
    plt.figure(figsize=(9, 5))
    sns.barplot(x=category_counts.values, y=category_counts.index)
    plt.title("Products by Category (Top 15)")
    plt.xlabel("Number of Products")
    plt.ylabel("Category")
    plt.tight_layout()
    plt.show()
```

```
Category chart skipped: no product data is available.
```

In [20…

```
if df.empty:
    print("Boxplot skipped: no product data is available.")
else:
    plt.figure(figsize=(7, 5))
    sns.boxplot(y=df["Price_Treated"])
    plt.title("Product Prices After Outlier Treatment")
    plt.ylabel("Treated Price")
    plt.tight_layout()
    plt.show()
```

```
Boxplot skipped: no product data is available.
```

SECTION 11: Machine Learning

11.1 Preparing the Data

The target is `Price_Category`. The price itself is excluded from the input features to avoid directly giving the model the answer.

In [21…
```
feature_columns = ["Description_Length", "Product_Name_Length", "Category
model_trained = False

if df.empty or "Price_Category" not in df.columns:
    print("Machine learning skipped: no prepared product records are avai
else:
    ml_df = df.copy()
    X = ml_df[feature_columns].replace([np.inf, -np.inf], np.nan)
    X = X.fillna(X.median()).fillna(0)
    y = ml_df["Price_Category"].astype(str)

    print("Target class counts:")
    display(y.value_counts().to_frame("Count"))

    class_counts = y.value_counts()
    if len(ml_df) < 10 or y.nunique() < 2 or class_counts.min() < 2:
        print("Not enough data or target-class variation for a useful tra
    else:
        stratify_target = y if class_counts.min() >= 2 else None
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=stratify_targe
        )
        model = RandomForestClassifier(n_estimators=100, random_state=42,
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        model_trained = True
        print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
        print("\nClassification report:")
        print(classification_report(y_test, y_pred, zero_division=0))
```

```
Machine learning skipped: no prepared product records are available.
```

11.2 Confusion Matrix

The confusion matrix is displayed only if a model was successfully trained.

In [22…
```
if model_trained:
    labels = sorted(pd.Series(y).unique())
    cm = confusion_matrix(y_test, y_pred, labels=labels)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
```

```
                xticklabels=labels, yticklabels=labels)
    plt.title("Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.show()
else:
    print("Confusion matrix skipped because the model was not trained.")
```

```
Confusion matrix skipped because the model was not trained.
```

SECTION 12: Final Dataset and Verification

12.1 Reviewing the Final Dataset

We review the final data after cleaning and feature engineering.

In [23…

```
print("Final dataset shape:", df.shape)
display(df.head())
```

```
Final dataset shape: (0, 15)
```

Product_Name	Price	Category	Description	Availability	Product_URL	Source	Price_Numer

12.2 Final Quality Check

We check missing values, duplicate records, and unique product URLs.

In [24…

```
print("Missing values by column:")
display(df.isna().sum().to_frame("Missing_Count"))
print("Duplicate rows:", df.duplicated().sum())
print("Unique product URLs:", df["Product_URL"].nunique() if "Product_URL
print("Final rows:", len(df))
```

```
Missing values by column:
```

	Missing_Count
Product_Name	0
Price	0
Category	0
Description	0
Availability	0
Product_URL	0
Source	0
Price_Numeric	0
Price_Treated	0
Description_Length	0
Product_Name_Length	0
Availability_Flag	0
Price_Category	0
Premium_Product	0
Category_Code	0

```
Duplicate rows: 0
Unique product URLs: 0
Final rows: 0
```

12.3 Saving and Downloading the Datasets

The raw and final datasets are saved separately. Run the download cells when you want to download them to your computer.

In [25…
```
df_raw.to_csv("/content/raw_ecommerce_data.csv", index=False)
df.to_csv("/content/final_ecommerce_dataset.csv", index=False)
print("Raw dataset saved:", os.path.exists("/content/raw_ecommerce_data.c
print("Final dataset saved:", os.path.exists("/content/final_ecommerce_da
print("Raw rows:", len(df_raw), "| Final rows:", len(df))
```

```
Raw dataset saved: True
Final dataset saved: True
Raw rows: 146 | Final rows: 0
```

In [28…
```
from google.colab import files
files.download("/content/raw_ecommerce_data.csv")
```

In [29…
```
from google.colab import files
```

```
files.download("/content/final_ecommerce_dataset.csv")
```

SECTION 13: Conclusion

Product information was collected from a public e-commerce test website and stored in a structured dataset. The data was checked, cleaned, and enriched with new features. Price patterns were visualized, and a basic classifier was attempted when the available data supported it.

The final number of unique products depends on the source pages that are accessible. The notebook does not duplicate or invent product records to meet the 200-row target. If fewer than 200 products are available, an additional approved data source is needed.